סיכום שיעור 2: Select SQL

קורס: מבוא לבסיסי נתונים

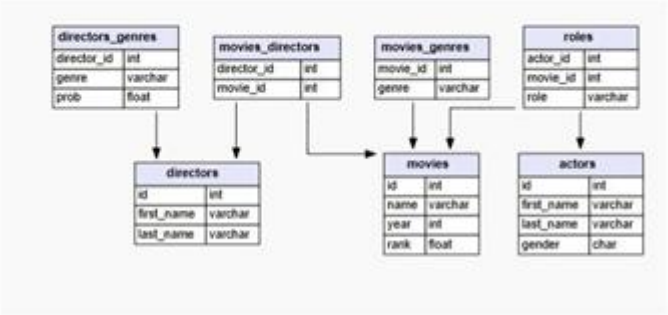
השיעור עוסק בהיכרות עם שפת ה-SQL במיקוד על פקודת ה-SELECT ומסד הנתונים שאיתו נעבוד בקורס.

להלן קישור למצגת השיעור

1. היכרות עם מבנה הנתונים שאיתו נעבוד בקורס (IMDB dataset): קישור לדאטה סט

- מאגר המלצות לסרטים אמריקאים מאוד גדול
- עברנו על הקשרים בין הטבלאות כפי שניתן לראות בתמונה:

IMDB dataset



פירוט אודות הטבלאות:

- movies – מכילה מידע על סרטים: id (מזהה סרט), name (שם הסרט), year (שנת יציאה), rank (דירוג).
- directors – מכילה מידע על במאים: id (מזהה), first_name (שם פרטי), last_name (שם משפחה).
- actors – מכילה מידע על שחקנים: id (מזהה), first_name (שם פרטי), last_name (שם משפחה), gender (מגדר).
- movies_directors – טבלת קישור בין סרטים לבמאים (קשר many-to-many): movie_id (מזהה סרט), director_id (מזהה במאי).

- roles – מציינת איזה שחקן שיחק באיזה סרט ובאיזה תפקיד: actor_id (מזהה שחקן), movie_id (מזהה סרט), role (תפקיד).
- movies_genres – קישור בין סרטים לז'אנרים (לסרט יכולים להיות כמה ז'אנרים): movie_id (מזהה סרט), genre (ז'אנר).
- directors_genres – מראה את הקשר בין במאים לז'אנרים כולל הסתברות: director_id (מזהה במאי), genre (ז'אנר), prob (הסתברות).

2. הסבר על שפת SQL ושימושיה

- SEQUEL הינה שפת שאילתות באנגלית מובנית, 1974.
- C נוצרה בשנת 1972, C++ בשנת 1985, Python בשנת 1991, Java בשנת 1995.
- מבוססת על תורת הקבוצות, מה שהופך אותה לאלגנטית וחזקה לניתוח נתונים.

4. הסבר על פקודת ה- SELECT

Select - The Superman of data analytics

```
select *
from imdb_ijs.movies
where name = 'Superman'
order by year;
```

- השאילתות הן הצהרתיות (Declarative) – מגדירות מה צריך, ולא איך לממש את זה.
- השאילתות מאוד מובנות (בעלות מבנה קבוע וברור). לכן חייבים לכתוב את השאילתה בסדר מסויים כדי שהיא תהיה תקינה.
- קישור לדוגמאות לשימוש בפקודת ה-SELECT ותרגילי הכיתה שנעשו

5. מבנה כללי לפקודת SELECT (בצורה פשוטה)

הרחבה על פקודת ה-SELECT

Select statement structure (simplified)

```
SELECT
    [ALL | DISTINCT | DISTINCTROW ]
     select_expr [, select_expr] ...
     [FROM table_references
     [WHERE where_condition]
    [GROUP BY {col_name | expr | position}
    [HAVING where_condition]
        [ORDER BY {col_name | expr | position}
     [ASC | DESC], ..]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
```

6. פונקציות

- ל-SQL יש "פונקציות רגילות" שמוסיפות יותר כוח לשאילתות.
- יש הרבה מאוד פונקציות, והן שימושיות למגוון רחב של שימושים ופעולות שנרצה לבצע.
- הן אינן המוקד של הקורס הזה.
- מצגת הסבר על פונקציות (עברנו עליה בביתה רק לצורך היכרות בסיסית עם הפונקציות) חשוב לדעת ש:
  - קיימות מעל 400 פונקציות כאשר רובן לא רלוונטיות לקרוס זה)
  - אין צורך לדעת את כל הפונקציות בע"פ
  - ניתן גם לכתוב פונקציות חדשות

7. אופרטורים, מחרוזות, אגרגציה וכו' (קישור למצגת שעברנו עליה)

- עסקנו בהיכרות עם אופרטורים שימושים לטובת זיהוי דפוסים בדאטה כמו LIKE, Exist/in ונוספים
- לטובת הקורס אין צורך לעשות שימוש ב- Regexp בשאילתות שנכתוב / נשתמש
- שוחחנו על פעולות שניתן לבצע על מחרוזות:

הכרנו את:

Concat – מחבר כמה מחרוזות למחרוזת אחת

Instr – מחפש מיקום של תת־מחרוזת בתוך מחרוזת

Length – מחזיר את אורך המחרוזת

Lower / Upper – הופך את הטקסט לאותיות קטנות / גדולות

Substr / Left / Right – מחלץ חלק מתוך מחרוזת

להלן דוגמאות שבוצעו בשיעור בנושא:

· ראינו באמצעות דרכים שונות ופונקציות שונות ניתן לחלץ את אותו מידע - לדוגמה חילוץ שמות שחקנים ששמם הפרטי ושם המשפחה שלהם מתחיל באותו אות הן באמצעות SUBSTRING והן באמצעות- LEFT.
להלן דוגמה לשימוש בשינהם:

*נציין שבמקרה זה שימוש ב-LEF פשוט יותר ולכן עדיף.

```
-- Actors whose first and last name start in the same letter

SELECT * FROM imdb_ijs.actors

WHERE LEFT(first_name, 1) = LEFT(last_name, 1);


-- Or:

select * from imdb_ijs.actors

where substring(first_name,1,1) = substring(last_name,1,1)
```

· עברנו על אופרטורים ה-IF ו-CASE המשמשות לכתיבת שאילתות עם תנאים מכוננים במספר דרכים.
ראינו שמאוד נוח ומומלץ להשתמש ב-CASE לטובת חלוקה לתנאים.

<u>הרחבה על CASE</u>:

מאפשר ליצור עמודה חדשה על בסיס תנאים, בדומה ל-IF / ELSE IF / ELSE בצורה ברורה יותר, כאשר END הוא פשוט הסוגר של כל בלוק ה-CASE
END = finishes the CASE logic

· להלן דוגמה שבוצעה בכיתה שממחישה את שני הדרכים:

```
-- Sherlock Holmes with production periods

SELECT *
```

```
, year
, IF(year > 2000, 'New',
IF(year > 1939, 'WWII',
IF(year < 1914, 'Pre WWII', 'Other')
)
) AS period
FROM movies
WHERE name LIKE '%Sherlock Holmes%'
ORDER BY year DESC;
-- or
select
year as production_year, name
, case
when year < 1920 then 'Ancient'
when year < 1939 then 'Pre WWII'
when year > 1990 then 'New (for this dataset)'
else 'In between'
end as age
from movies
where
name like '%Sherlock Holmes%'
order by year desc, name;
```

- ניתן לראות כי במקרה זה כי שימוש ב- case מפשט את הפתרון. CASE הרבה יותר קריא, מסודר וקל לשליטה כשיש כמה תנאים

8. תרגול בביתה

- במהלך השיעור תרגלנו דוגמאות רבות ומגוונות לשימוש בפקודת ה-SELECT. בקישור הזה נמצאים כלל הפתרונות לתרגילים שבוצעו בביתה, המופיעים בשתי השקופיות האחרונות במצגת של השיעור.
- מצרפת בנספח 1 (בסוף הסיכום) את כלל הדוגמאות שפתרנו בביתה על מנת שיופיעו גם בסיכום השיעור (מרוכזים גם בקישור שצירפתי לעיל)

9. חיבור בין טבלאות - JOIN

אנחנו נלמד על join באופן מלא בשיעור שלישי. כרגע, נציץ על שימוש ביותר מטבלה אחת.

JOIN = חיבור בין טבלאות לפי עמודות משותפות (מפתחות), כדי להביא מידע שלא נמצא בטבלה אחת בלבד.

- תחביר כללי:

```
SELECT ...
FROM table1 t1
JOIN table2 t2
ON t1.column = t2.column
```

- להלן קישור למבנה כללי של JOIN
- דוגמה שבוצעה בביתה בנושא:

```
select first_name, last_name, genre
from directors_genres as dg
join
directors as d
```

```
on

;dg.director_id = d.id
```

*במסגרת נושא זה שוחחנו על שימוש ב-as לצורך הגדרת שם לבחירתנו לטבלה (לצורך נוחות) – ניתן לראות שימוש בכך בדוגמה שלהלן.

10. מספר נקודות / הערות נוספות שדיברנו עליהם בשיעור:

- כשאנחנו כותבים use imdb_ijs – זה מאפשר לנו לכתוב רק את שם הטבלה, במקום לכתוב בכל שאילתה from imdb_ijs. לדוגמה:

```
select *

from movies

where name = 'Superman'
```

במקום זה:

```
select *

from imdb_ijs.movies

where name = 'Superman'
```

- כברירת מחדל, SQL אינה רגישה לאותיות גדולות/קטנות (case sensitive)
- שוחחנו על אמינות הנתונים וטעויות שיכולות לפגוש אותנו בדאטה, כמו למשל שימוש בראשי תיבות במקום שם פרטי, שימוש במספרים כשמות וכו', להלן דוגמאות:

first_name
99
A
A.
Ab
AC

first_name
'E.'
'J.G.' Jim
'J.J.' Joseph Otieno
'M.'
'T.J.' Jimmy

- כמו כן, עברנו על המטלה הראשונה של שיעורי הבית. הסבר למטלה מופיע בקובץ הבא. אין חובת הגשה למטלה אך מומלץ לבצע אותה כדי לחזור על החומר (תיחשב כמטלת בונוס למי שיגיש אותה). כמו כן, צורת ההגשה הרצויה מפורטת בעמוד הראשון של הקובץ שצורף בנוגע למטלות הביתה. לא לשכוח להגיש שני קבצים בעת ההגשה (קובץ PDF וקובץ SQL תואם עם ת"ז של הסטודנט לפי הפורמט הנדרש).

נספח 1 - תרגילי ביתה:

```
select *
from imdb_ijs.movies
where name = 'Superman'
order by year;
```

```
select *
from imdb_ijs.movies
where name = 'frozen'
order by year;
```

```
select *
from actors
where first_name = 'Adam'; #Actor by choise
```

```
# Actors named Marilyn
SELECT * FROM imdb_ijs.actors
WHERE first_name = 'Marilyn';
```

```
# Directors named Hitchcock
SELECT * FROM imdb_ijs.directors
WHERE last_name = 'Hitchcock';
```

```
#Actors whose first and last name start in the same letter
SELECT * FROM imdb_ijs.actors
WHERE LEFT(first_name, 1) = LEFT(last_name, 1);
-- Or:
select * from imdb_ijs.actors
where substring(first_name,1,1) = substring(last_name,1,1);


# Sherlock Holmes with production periods
select
year as production_year, name
, case
when year < 1920 then 'Ancient'
when year < 1939 then 'Pre WWII'
when year > 1990 then 'New (for this dataset)'
else 'In between'
end as age
from movies
where
name like '%Sherlock Holmes%'
order by year desc, name;
-- or
SELECT *
```

```
, year
, IF(year > 2000, 'New',
IF(year > 1939, 'WWII',
IF(year < 1914, 'Pre WWII', 'Other')
)
) AS period
FROM movies
WHERE name LIKE '%Sherlock Holmes%'
ORDER BY year DESC;


# All the movies from the eighties
select year as production_year, name
from movies
where year between 1980 and 1989
-- additional options:
-- where year >= 1980 and year < 1990
-- where year like "198%"
order by year, name;
```

·<u>הסבר על האופציות השונות</u>:

המטרה: להחזיר סרטים משנות ה־80 (1980–1989)

where year between 1980 and 1989

המשמעות: כולל גם 1980 וגם 1989 (year ≤ 1989 ≥ 1980)

יתרונות: ברור וקריא, מתאים לטווחים סגורים

1990 > and year 1980 =< where year

המשמעות: כולל 1980 אבל לא כולל 1990 (1990 > year ≥ 1980)

יתרונות: מדויק, פחות טעויות גבול

where year like "198%"

המשמעות: כל ערך שמתחיל ב־"198"

הערה חשובה: עובד נכון רק אם year הוא טקסט (STRING)

```
# All movies that have ‘star’, insensitive to case, in their name
select * from movies
where lower(name) like '%star%';
-- or: we can use instr()
```

```
SELECT *
FROM movies
WHERE INSTR(LOWER(name), 'star') > 0;
```

·הסבר על שימוש ב- instr:

instr היא פונקציה שמחפשת מחרוזת בתוך מחרוזת אחרת ומחזירה את המיקום שלה. INSTR(string, substring)

LOWER(name) - הופך את שם הסרט לאותיות קטנות

INSTR(...) - מחפש את המחרוזת 'star' בתוך השם

אם נמצא - מחזיר מיקום (0 <)

אם לא נמצא - מחזיר